Foundations of Data Science for Biologists

Introduction to Unix

BIO 724D

2026-JAN-13

Instructors: Greg Wray and Paul Magwene

Learning goals for today

Learn how to navigate within the Unix file system

Learn how to work with files from the command line

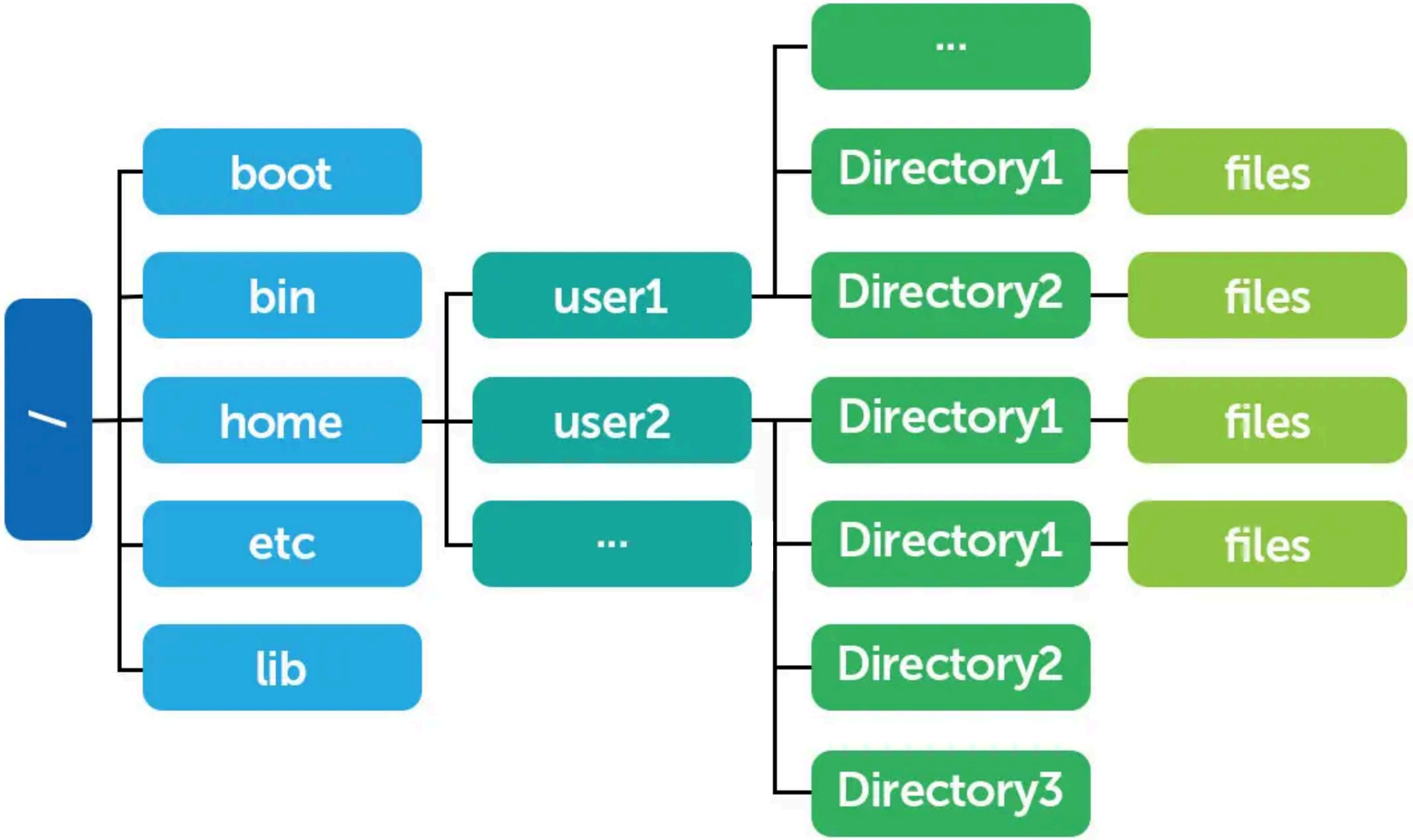
Learn how to get help for commands

Learn how to redirect input and output

Learn how to manipulate files using basic Unix utilities

Navigating the Unix file system

The Unix file system



root OS users user directories and files

Paths

Paths describe the location of a file or directory

Relative path: a description in relation to your current working directory

- To navigate “down” (deeper, away from the root) provide the directory name
- To navigate “up” use `..`
- Useful for working interactively; saves a lot of typing and avoids mistakes

Absolute path: a complete and unambiguous description starting at the root

- Points to the same file or directory regardless where your working directory is
- There is only one absolute path to a given file or directory
- Always begins with `/`
- Useful for scripts and programs that might be run from multiple locations

Basic navigation

To change directory, use **cd** and provide a path:

`cd data`	relative path to sub-directory called `data`
`cd data/january/08`	relative path to sub-sub-sub-directory called `08`
`cd /opt/R`	absolute path to directory `R`

Several short-cuts are very convenient:

`cd ~`	navigate to your home directory
`cd /`	navigate to the root of the file system
`cd ..`	navigate one level “up” in the file system
`cd ../..`	navigate two levels “up” in the file system
`cd -`	navigate to the previous directory

You can always ask where you are:

`pwd`	short for **p**rint **w**orking **d**irectory

Basic file operations in Unix

With great power…

Unix assumes you know what you are doing

Enormous power, but minimal supervision (= great responsibility)

Most file operations are executed without safety checks

However, you can use flags to add checks to some file operations

There is no undo for file operations, no recycle/trash folder for deleted files

Always work with duplicates of original data and code (not originals)

There is typically no feedback unless an error occurs

Check results to confirm (e.g., with `ls` and `pwd`)

General points about file operations

Commands assume the current directory unless otherwise specified

- To specify a different directory, provide the path before the file name
- Alternatively, navigate to that directory and perform the operation

Commands do not affect sub-directories unless otherwise specified

- To affect sub-directories, use the -R flag (Recursion) with commands
- Comprehensively visits every sub-directory, sub-sub-directories, etc.

Many commands allow multiple inputs

- Allows you to create/rename/move/delete multiple files/directories at once
- Separate name of files/directories with spaces; any number is allowed

General points about command syntax

Most commands use the same syntax

`command options file`	if only 1 file is involved
`command options file file`	if multiple files are involved (e.g., renaming)

Precede options with a dash; separate file names with a space

`ls -R data`	list contents of `data` and its sub-directories
`cat -n file1.txt file2.txt`	output merged content of files with line numbers

Options can be combined in any order and/or presented separately

`ls -l -a`	list all directory contents in long form
`ls -al`	same as above; order does not matter

Creating empty files

Use `touch` to create an empty file or to update the timestamp on an existing file

- If the specified file(s) does not exist, creates an empty file
- If the specified file(s) exists, simply updates the timestamp of most recent access

`touch chapter_1.txt`	creates an empty file
`touch data/chapter_1.txt`	creates an empty file in directory `data`
`touch chapter_{01..10}.txt`	creates 10 empty files (`chapter_01.txt,…`)

Notes:

- You can create multiple files at once and can create files in other locations
- Unix keeps separate timestamps for each file's creation and most recent access
- The last command is an example of an **expansion** (more on these later)

Deleting files

Use `rm` to delete one or more files (name is a contraction of **rm**ove)

`rm temp_1.txt`	removes a single file
`rm temp_1.txt thesis/chap_4.txt`	removes two files, one from `thesis`
`rm temp_*.txt`	removes all files matching the pattern
`rm *.txt`	removes all files with extension `.txt`

Notes:

- The shell will return an error message if the file does not exist
- Multiple files can be specified (separate with spaces)
- Files at other locations can be specified (provide a path)
- The last two commands are examples of wildcards or **globbing** (more on this later)

Copying files

Use `cp` to make copies of files (name is a contraction of **c**o**p**y)

`cp temp_01.txt temp_backup.txt`	makes a copy in the same directory
`cp temp_01.txt data`	makes a copy in `data`, using same name
`cp temp_01.txt data/test.txt`	makes a copy in `data`, using different name
`cp temp_01.txt temp_02.txt data`	copies 2 files to `data`, using same names

Notes:

- First argument is original file, second argument is new file
- New file will placed in current directory unless a path is provided
- New file name must be different if copying into current directory
- Multiple files can be copied to a single different location (last example)
- Also works with directories (use the `-R` option to copy any enclosed directories)

Creating directories

Use `mkdir` to create new directories (name is a contraction of **m**a**k**e **dir**ectory)

`mkdir data`	creates a directory in the current directory
`mkdir ~/data/my_project`	creates a directory at the specified location
`mkdir practice temp`	creates two new directories
`mkdir -p thesis/chapter_01`	creates nested directories

Notes:

- The shell will return an error message if the directory already exists
- Multiple directories can be created at once, but they will not be nested by default
- To create nested directories in a single step, the `-p` option is required

Deleting directories

Use `rmdir` to delete an existing **empty** directory

```
rmdir practice
rmdir data/week_02 data
```

deletes a directory

deletes two directories in the order given

Notes:

- The shell will return an error if the directory does not exist or if it contains any files
- `rm` deletes directories containing files without at warning (not recommended)
- Delete nested directories *before* deleting parent directories (second example)

Moving and renaming files/directories

Use `mv` to move and/or rename files or directories (name is a contraction of **m**o**v**e)

`mv temp_01.txt data`	moves file to different location
`mv temp_01.txt perm_01.txt`	renames file without moving it
`mv temp_01.txt data/perm_01.txt`	moves and renames file
`mv data data_original`	renames directory

Notes:

- First argument is file/directory, second argument is new name or location
- File/directory can be renamed without/while moving (second example)
- Works with directories (folders) and files

Tips for copying, moving, and renaming files

Be aware! The shell will:

- **overwrite** existing files and directories if they have the same name
- give you **no warning** that it did this

`rm` has options that provide a layer of security:

```
rm -i temp_01.txt
rm -v temp_??.txt
```

prompts y/n before deleting

reports on each file successfully deleted

`cp` and `mv` have options that provide a layer of security:

```
mv -i temp_01.txt /data
mv -n temp_01.txt data_01.txt
```

prompts y/n before overwriting

prevents overwriting

Viewing file contents

Viewing the contents of text files

Use the `cat` command to view file contents (name is short for con**cat**enate)

`cat temp_1.txt`	outputs contents to stdout
`cat temp_1.txt temp_2.txt`	outputs merged contents, in order given
`cat -n temp_1.txt`	adds line numbers to output

Notes:

`cat` is very useful in pipes for passing the contents of a file to the next command

Viewing the beginning or end of files

Use the `head` and `tail` commands for a quick view of file contents

```
head temp_1.txt
head -n40 temp_1.txt
tail -n15 temp_1.txt
```

displays first 10 lines

displays first 40 lines

displays last 15 lines

Notes:

- The `-n` option lets you control how much to show (default = 10)
- These commands are particularly helpful when files are large

Viewing a file in full-screen mode

Use the `less` command

```
less temp_1.txt
less temp_1.txt temp_2.txt
```

opens a full-screen browser

queues two files for viewing

Navigation:

<space> or Z	scroll one screen down
W	scroll one screen up
<arrow up/down>	scroll one line up or down
/	search for a pattern (plain text or regex)
H or h	display help
Q or q	quit and return to the command line

Getting help

Getting help with commands

Use `man` to get detailed help about a command; opens in `less`

```
man cp
```

shows help for `cp`; press q to exit

Use `info` to get even more detailed help; not provided by some shells (e.g., zsh)

```
help cp
```

shows help for `cp`; press q to exit

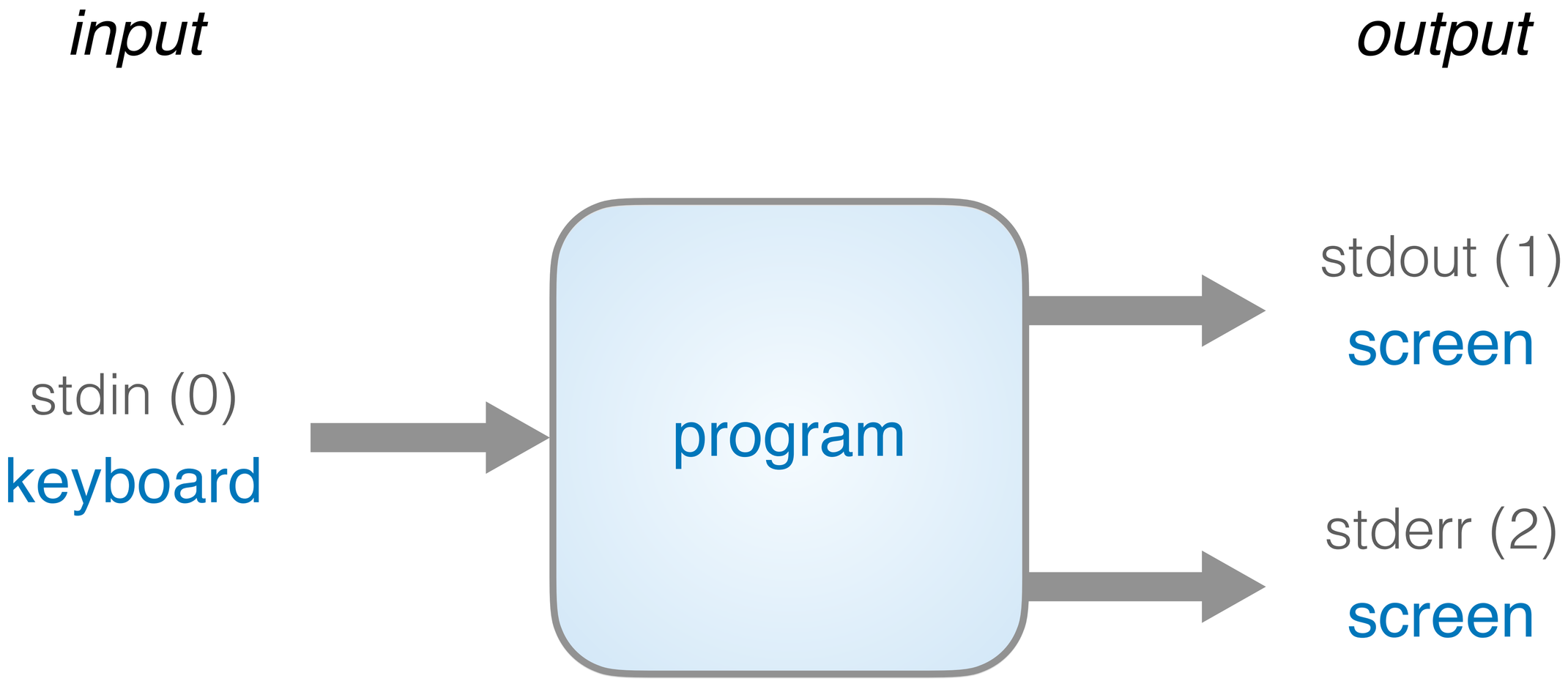
Use the `--help` option for brief help; displays at command line; does not execute command

```
cp --help
```

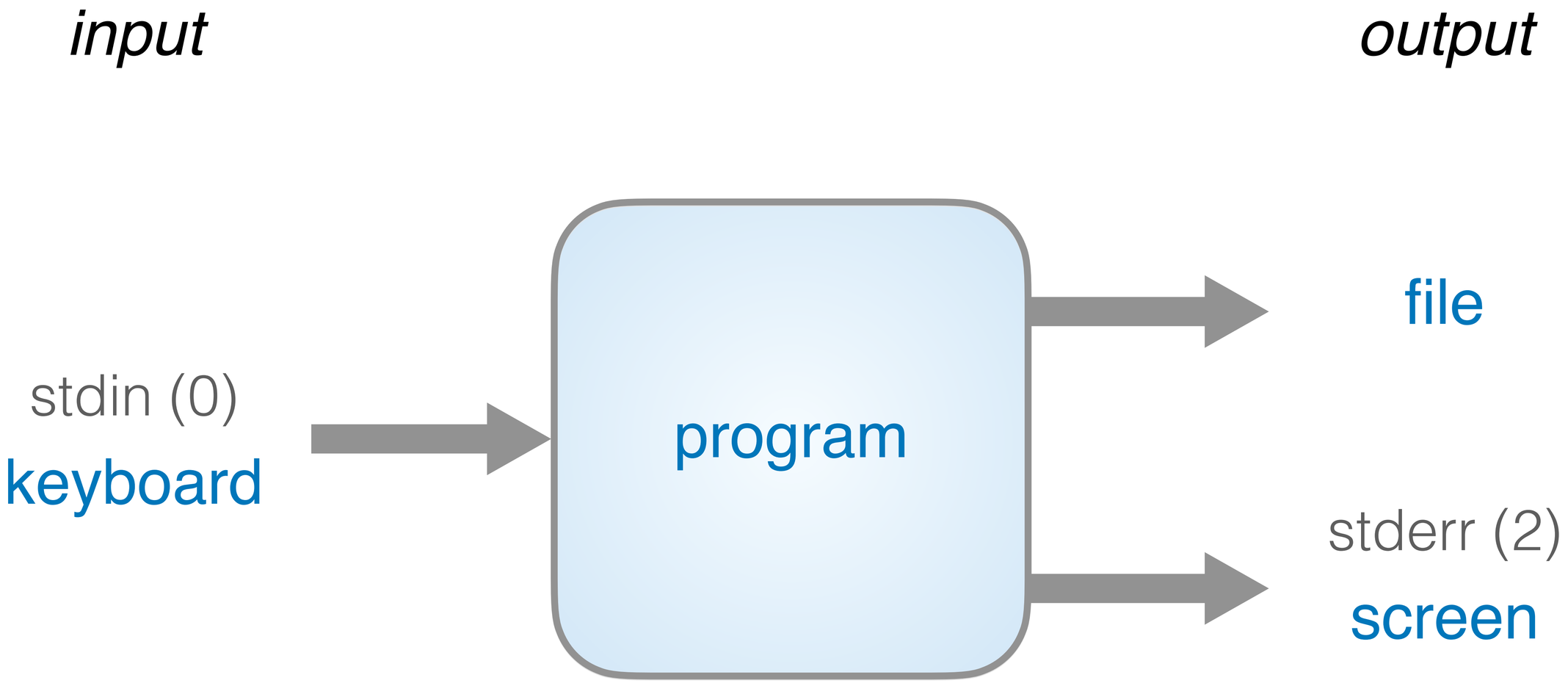
shows brief help for `cp`

Redirection and pipes

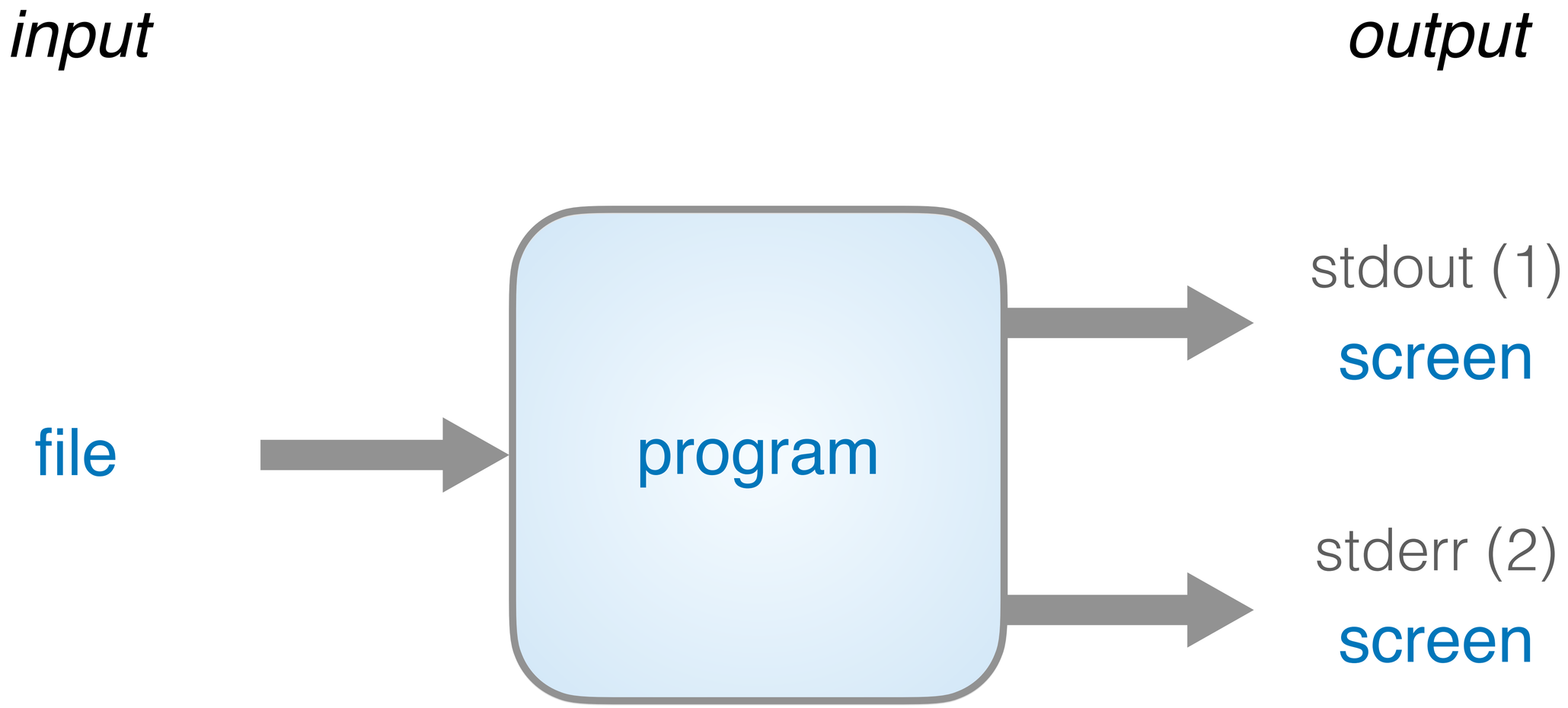
Standard input and output: defaults



Redirecting output to a file



Redirecting input from a file



Changing the default streams

Redirection changes the defaults streams:

`>`	directs stdout to a file: create new or over-write existing
`>>`	directs stdout to a file: create new or append to existing
`2>`	directs stderr to a file: create new or over-write existing
`2>>`	directs stderr to a file: create new or append to existing
`<`	replaces stdin with file contents (often clearer to pipe from `cat`)

Piping is a special case of redirection:

`|`	directs stdout from one command to another command or program

Writing strings to a file

Use `echo` to write strings

```
echo hello world
echo hello world >> temp_1.txt
echo "hello again" >> temp_1.txt
```

outputs `hello world` at the terminal

creates a file containg `hello world`

appends `hello again` to the file

Notes:

- Quotes not needed unless inserting white space, escape characters, or variables
- Each echo command adds at least one new line by default
- `echo` can be surprisingly useful as you get familiar with Unix

Writing program or command output to a file

Use the `>` and `>>` operators to **redirect** output to a file instead of the terminal

```
echo -n "directory " > t3.txt
pwd >> t3.txt
echo -n "on " >> t3.txt
date >> t3.txt
ls -l >> t3.txt
```

starts a small text file

appends the path to the directory

starts a new line

appends a timestamp

outputs the directory list to the file

Notes:

- Use quotes to insert a trailing space
- Use `-n` option with `echo` to remove the trailing next line character

Exercise

Create a folder called `thesis_project_1`

Create a subfolder within `thesis_project_1` called `original_data`

Navigate to the new subfolder

Create 10 text files called called `data_01.txt` through `data_10.txt`

Delete the file `data_05.txt`

Without changing location, insert a listing of your home directory into `data_01.txt`

View the contents of `data_01.txt` with line numbers

Tip: at each step, use `ls`, `pwd`, and/or `cat` to check your results!

Example of piping

```
cat VoyageOfTheBeagle.txt |
tr [:space:] "\n" |          # convert whitespace to return
tr -d [:punct:] |            # delete all punctuation
tr [:upper:] [:lower:] |     # make everything lower case
grep -v '^$' |               # delete blank lines
sort |                       # uniq requires sorted data
uniq -c |                    # find uniq lines and count them
sort -k1 -r -n |             # sort on counts
less                         # visualize
```